

Estudo Comparativo de Implementações do Protocolo TCP

Trabalho Prático — Redes de Computadores Variantes analisadas: TCP Tahoe · TCP Reno · TCP NewReno · TCP Vegas · TCP SACK

Sumário

1. Introdução ao Controle de Congestionamento no TCP
2. Descrição Individual das Variantes
3. Tabela Comparativa
4. Análise de Cenários
5. Conclusão
6. Referências

1. Introdução ao Controle de Congestionamento no TCP

1.1 O papel do controle de congestionamento

O TCP (*Transmission Control Protocol*, RFC 793, atualizado pela RFC 9293) oferece um serviço de transporte **confiável, orientado a conexão e ordenado** sobre uma rede IP que, por si só, é apenas de melhor esforço — a rede pode descartar, duplicar, atrasar ou reordenar pacotes. Para entregar seus dados de forma íntegra, o TCP usa números de sequência, reconhecimentos (ACKs) e retransmissões.

É preciso separar dois mecanismos distintos:

- **Controle de fluxo** protege o *receptor*: a janela anunciada (`rwnd`) impede que um emissor rápido sobrecarregue o buffer de um receptor lento.
- **Controle de congestionamento** protege a *rede*: uma janela de congestionamento (`cwnd`) limita quantos dados o emissor coloca "em voo" antes de receber ACKs, evitando saturar os roteadores no caminho.

A quantidade de dados não reconhecidos que o emissor pode ter em trânsito é $\min(\text{cwnd}, \text{rwnd})$. O controle de congestionamento é, essencialmente, o algoritmo que ajusta `cwnd` ao longo do tempo.

A motivação histórica foi concreta: em outubro de 1986 a Internet sofreu os primeiros **colapsos de congestionamento** (*congestion collapse*), em que a vazão útil despencava a quase zero porque a rede ficava saturada de retransmissões. Em resposta, Van Jacobson apresentou em 1988 o artigo seminal "*Congestion Avoidance and Control*", que introduziu os pilares ainda usados hoje: **slow start**, **congestion avoidance**, **fast retransmit** e a heurística **AIMD** (*Additive Increase, Multiplicative Decrease*) — crescer devagar de forma aditiva e recuar de forma multiplicativa (pela metade) ao detectar perda. A perda de pacotes é interpretada como o **sinal implícito de congestionamento**.

Os parâmetros centrais que aparecem em todas as variantes são:

- **cwnd** — janela de congestionamento;
- **ssthresh** — limiar que separa a fase de *slow start* (crescimento exponencial) da fase de *congestion avoidance* (crescimento linear);
- **MSS** — *Maximum Segment Size*, a unidade em que a janela é medida.

1.2 Por que surgiram diferentes variantes

O algoritmo original (Tahoe) resolveu o colapso, mas era conservador demais: a cada perda ele zera a janela e recomeça do *slow start*, desperdiçando vazão. Cada variante seguinte ataca uma limitação específica:

- **Reno** — recuperar-se de uma perda isolada *sem* zerar a janela (Fast Recovery).
- **NewReno** — recuperar **múltiplas perdas** na mesma janela sem cair em timeout, usando apenas informação do emissor.
- **Vegas** — mudar o paradigma: detectar congestionamento pelo **aumento do atraso (RTT)** *antes* que a perda aconteça, em vez de reagir à perda.
- **SACK** — mudar a **informação de reconhecimento**: dizer ao emissor exatamente *quais* segmentos chegaram, permitindo retransmitir só os buracos.

Além disso, redes com características extremas — **alto produto banda-atraso** (satélite, enlaces de longa distância) e **perdas não relacionadas a congestionamento** (redes sem fio, 4G/5G) — expuseram fraquezas das variantes baseadas apenas em perda, motivando abordagens seletivas e baseadas em atraso.

2. Descrição Individual das Variantes

2.1 TCP Tahoe (1988)

- **Mecanismo principal.** Combina três algoritmos de Jacobson: **Slow Start** (**cwnd** dobra a cada RTT, crescimento exponencial, até atingir **ssthresh**), **Congestion Avoidance** (a partir de **ssthresh**, **cwnd** cresce ~ 1 MSS por RTT, aditivo) e **Fast Retransmit** (ao receber **3 ACKs duplicados**, retransmite o segmento perdido imediatamente, sem esperar o timeout).
- **Tratamento de perdas.** Tahoe reage **igual** a qualquer sinal de perda — seja timeout, seja 3 ACKs duplicados: faz **$ssthresh = FlightSize/2$** , coloca **$cwnd = 1$ MSS** e **reinicia o slow start**.
- **Fast Recovery? Não.** Mesmo após um fast retransmit, a janela volta a 1 MSS.
- **Vantagens.** Simples; resolveu o problema do colapso de congestionamento; base conceitual de todas as variantes seguintes.
- **Desvantagens.** Zerar **cwnd** a cada perda é caríssimo em vazão, sobretudo em enlaces de alto produto banda-atraso; a reconstrução da janela via slow start é lenta.
- **Histórico.** Surgiu na distribuição **4.3BSD "Tahoe"** (1988), implementando os algoritmos do artigo de Jacobson (SIGCOMM 1988). Não teve RFC próprio na época; os algoritmos foram depois padronizados nas RFCs 2001, 2581 e 5681.

2.2 TCP Reno (1990)

- **Mecanismo principal.** É o Tahoe **acrescido do Fast Recovery**. Ao detectar 3 ACKs duplicados, faz $ssthresh = FlightSize/2$, retransmite (fast retransmit) e entra em Fast Recovery: $cwnd = ssthresh + 3 \cdot MSS$, "inflando" $cwnd$ a cada novo ACK duplicado (cada duplicado indica que um pacote saiu da rede). Quando chega um ACK que reconhece dados novos, faz $cwnd = ssthresh$ (desinfla) e segue direto em Congestion Avoidance — **sem voltar ao slow start**.
- **Tratamento de perdas. 3 ACKs duplicados** → fast retransmit + Fast Recovery (a janela cai pela metade, não a 1). **Timeout** → comportamento do Tahoe: $cwnd = 1$ e slow start.
- **Fast Recovery? Sim** — foi a variante que o introduziu.
- **Vantagens.** Recupera-se muito melhor que o Tahoe de uma **perda isolada** por janela; preserva o "ACK clocking" e mantém vazão mais alta.
- **Desvantagens.** Foi projetado supondo **uma perda por janela**. Quando há **múltiplas perdas na mesma janela de transmissão**, o Reno tende a reduzir $cwnd$ várias vezes seguidas ou a cair em **timeout**, degradando bruscamente o desempenho.
- **Histórico.** Distribuição **4.3BSD "Reno"** (1990). Padronizado na **RFC 2001** (1997), depois **RFC 2581** (1999) e **RFC 5681** (2009, *TCP Congestion Control*, atualmente em vigor).

2.3 TCP NewReno (1999–2012)

- **Mecanismo principal.** Mantém o Reno, mas **corrige o Fast Recovery** para lidar com **múltiplas perdas por janela** usando o conceito de **ACK parcial** (*partial ACK*). Durante o Fast Recovery, um ACK que reconhece *parte* — mas não *todos* — os dados que estavam em voo ao entrar em recuperação sinaliza que ainda há outro segmento perdido. O NewReno então retransmite o próximo buraco e **permanece** em Fast Recovery, só saindo quando todos os dados do "ponto de recuperação" forem reconhecidos.
- **Tratamento de perdas. 3 ACKs duplicados** → fast retransmit + Fast Recovery com tratamento de ACKs parciais (recupera vários segmentos sem sair da fase de recuperação). **Timeout** → slow start.
- **Fast Recovery? Sim**, em versão aprimorada.
- **Vantagens.** Recupera múltiplas perdas por janela **sem depender de timeout** e, crucialmente, **sem exigir suporte do receptor** — é uma mudança só no emissor (diferente do SACK).
- **Desvantagens.** Recupera **cerca de uma perda por RTT** (uma retransmissão por ACK parcial); com muitas perdas, ainda fica lento. Como não usa SACK, o emissor **não sabe quais** segmentos faltam — infere um por vez.
- **Histórico.** **RFC 2582** (1999), revisada pela **RFC 3782** (2004) e, por fim, **RFC 6582** (2012, *The NewReno Modification to TCP's Fast Recovery Algorithm*).

2.4 TCP Vegas (1994)

- **Mecanismo principal.** Abordagem **baseada em atraso** (*delay-based*) e **proativa**. Em vez de esperar a perda, o Vegas monitora o RTT para inferir o enfileiramento. Ele compara a vazão **esperada** ($cwnd / BaseRTT$, onde $BaseRTT$ é o menor RTT observado) com a vazão **real** ($cwnd / RTT_{atual}$) e calcula $Diff = Esperada - Real$. Se $Diff < \alpha$, aumenta $cwnd$; se $Diff > \beta$, diminui; entre α e β , mantém — mirando manter **poucos pacotes na fila**. Também detecta perdas mais cedo, checando o RTT já no **primeiro** ACK duplicado (com timestamps finos), sem esperar por três.

- **Tratamento de perdas.** O objetivo é **evitar** a perda regulando `cwnd` pelo RTT; quando ela ocorre, a detecção é mais rápida que no Reno (retransmite ao ver o primeiro/segundo duplicado se o tempo do segmento já excedeu o limite).
- **Fast Recovery?** Usa recuperação no estilo Reno, mas com **retransmissão baseada em temporização mais refinada**; o diferencial não é o Fast Recovery e sim a *prevenção* da perda.
- **Vantagens.** Menos perdas, **menor atraso de enfileiramento** e menos retransmissões; excelente aproveitamento quando é o fluxo dominante no enlace.
- **Desvantagens.** **Injustiça ao competir** com fluxos baseados em perda (Reno/CUBIC): o Vegas recua ao ver o RTT subir, enquanto os outros continuam enchendo o buffer — e assim o Vegas "cede" banda. É sensível a erros de estimativa do `BaseRTT` (por exemplo, quando a rota muda) e a reordenações.
- **Histórico.** Proposto por **Brakmo, O'Malley e Peterson**, "*TCP Vegas: New Techniques for Congestion Detection and Avoidance*" (SIGCOMM 1994; versão estendida no IEEE JSAC, 1995). **Não foi padronizado em RFC** — é uma contribuição de pesquisa, mas influente: é ancestral conceitual de algoritmos modernos baseados em atraso/taxa, como o **BBR**.

2.5 TCP SACK — *Selective Acknowledgment* (1996)

- **Mecanismo principal.** Rigorosamente, o SACK **não é um algoritmo de congestionamento**, e sim uma **opção do TCP** que muda *a informação de reconhecimento*. Com o SACK negociado no handshake, o receptor informa, via blocos SACK, **quais faixas não contíguas** já recebeu. Assim o emissor sabe **exatamente quais** segmentos faltam e retransmite **somente os buracos**. Isso é combinado a um algoritmo de recuperação (por exemplo, a recuperação conservadora baseada em SACK da RFC 6675), que usa uma estimativa de dados em trânsito (a variável *pipe* e um *scoreboard*) para reenviar **várias perdas em um único RTT**.
 - **Tratamento de perdas.** Com a informação dos blocos SACK, o emissor identifica múltiplos buracos por janela e faz retransmissão **seletiva** de vários segmentos por RTT. **Timeout** ainda leva a `cwnd = 1` e slow start.
 - **Fast Recovery? Sim** — recuperação baseada em SACK, mais eficaz que a do NewReno para múltiplas perdas.
 - **Vantagens.** Recuperação **muito mais eficiente** de múltiplas perdas por janela (não gasta ~1 RTT por perda); ganho expressivo em enlaces de alto produto banda-atraso e em redes com perdas. A extensão **D-SACK** (RFC 2883) ainda permite detectar retransmissões espúrias.
 - **Desvantagens.** Exige **suporte dos dois lados** (negociação *SACK-Permitted*); acrescenta complexidade e estado no emissor (o *scoreboard* de blocos).
 - **Histórico.** **RFC 2018** (1996, *TCP Selective Acknowledgment Options*); **RFC 2883** (2000, D-SACK); a recuperação baseada em SACK foi especificada na **RFC 3517** (2003) e atualizada pela **RFC 6675** (2012). Hoje o SACK é habilitado por padrão nos principais sistemas operacionais.
-

3. Tabela Comparativa

Aspecto	Tahoe	Reno	NewReno	Vegas	SACK
Deteção de perda	Timeout ou 3 ACKs dup. → tratados igual	Timeout; 3 ACKs dup.	Timeout; 3 ACKs dup.	Aumento de RTT (antecipada) + dup. ACK precoce	3 ACKs dup. + blocos SACK identificam os buracos
Recuperação de perda	cwnd=1 + slow start (sempre)	Fast Recovery p/ 1 perda /janela; timeout → slow start	Fast Recovery com ACKs parciais (várias perdas, ~1/RTT)	Ajuste proativo de cwnd ; retransmissão baseada em tempo	Retransmissão seletiva ; várias perdas por RTT
Usa ACKs seletivos?	Não	Não	Não	Não	Sim (essência da variante)
Eficiência em redes com atraso (alto BDP)	Baixa (zera a janela)	Baixa/Média (sofre com múltiplas perdas)	Média (recupera 1 perda/RTT)	Média/Alta se sozinho; injusta ao competir	Alta (recupera sem gastar 1 RTT por perda)
Fast Recovery	Não	Sim	Sim (aprimorado)	Estilo Reno (foco em evitar perda)	Sim (baseado em SACK)
Sinal de congestionamento	Perda	Perda	Perda	Atraso (RTT)	Perda (com info seletiva)
Precisa de suporte do receptor?	Não	Não	Não	Não	Sim (negociação SACK)
Implementação em sistemas reais	Histórica (obsoleta)	Legada; base conceitual	Emissor-only; amplamente suportada	Nichos/pesquisa; inspira BBR	Padrão de fato — habilitada por default em Linux, Windows, macOS

4. Análise de Cenários

4.1 Em que tipo de rede cada variante se sai melhor?

- **Baixa latência e baixa perda (LAN, data center):** praticamente todas funcionam bem, pois há poucas perdas para recuperar. Reno/NewReno são suficientes; o **Vegas** brilha por manter filas curtas e latência baixa.
- **Alta perda (enlaces sem fio, redes congestionadas):** o **SACK** é o mais indicado, porque consegue recuperar **múltiplas perdas por janela** em poucos RTTs. Tahoe e Reno se saem mal (ver 4.2).
- **Banda limitada / buffers rasos:** o **Vegas** tende a ir bem porque **evita encher o buffer** e mantém a perda baixa — desde que não esteja competindo com fluxos agressivos baseados em perda, situação em que é prejudicado.
- **Alto atraso / alto produto banda-atraso (enlaces de longa distância, satélite):** o **SACK** é praticamente indispensável. Com RTT grande, cada timeout é catastrófico e recuperar "uma perda por RTT" (NewReno) é lento demais; a retransmissão seletiva do SACK preenche vários buracos por RTT. O NewReno é uma melhora sobre o Reno, mas fica atrás do SACK nesse cenário.

4.2 Qual variante tem pior desempenho em redes com perdas frequentes?

O **TCP Reno** é o que **degrada mais** quando há **múltiplas perdas na mesma janela**, situação típica de redes com perdas frequentes. Seu Fast Recovery foi desenhado para **uma** perda por janela: com várias, o Reno costuma sofrer reduções sucessivas de `cwnd` ou cair em **timeout** (voltando a `cwnd = 1`), colapsando a vazão. O **Tahoe** também vai mal — ele *sempre* reinicia o slow start —, mas seu comportamento é ao menos previsível; a fragilidade específica do Reno diante de perdas múltiplas por janela foi justamente o que motivou o NewReno e o SACK. Em resumo: **Reno é o pior caso prático em perdas frequentes**, com o Tahoe logo atrás por ser conservador demais.

4.3 Qual é a mais adequada para redes modernas (4G/5G, satélite)?

Entre as cinco variantes, o **TCP SACK** é o mais adequado para redes móveis e satelitais. Essas redes combinam dois fatores hostis às variantes clássicas baseadas apenas em perda: **perdas não relacionadas a congestionamento** (erros de rádio, *handovers*) e **alto produto banda-atraso**. O SACK lida bem com ambos, pois identifica e retransmite exatamente os segmentos perdidos sem exigir um RTT por perda nem interpretar toda perda como sinal de reduzir drasticamente a janela.

Vale registrar o contexto prático: a Internet moderna não usa mais Reno "puro". O padrão de fato hoje é **SACK combinado com um algoritmo de janela mais moderno**, como o **CUBIC** (RFC 8312, padrão no Linux) ou o **BBR** (baseado em taxa/atraso, herdeiro conceitual das ideias do Vegas). Dentro do escopo deste trabalho, porém, o **SACK** é a resposta correta — e, não por acaso, é o componente presente em essencialmente todas as pilhas atuais.

5. Conclusão

5.1 Resumo das descobertas

A evolução das variantes conta uma história clara de refinamentos sucessivos:

- **Tahoe** estabeleceu o alicerce (slow start, congestion avoidance, fast retransmit) e conteve o colapso de congestionamento, mas paga caro ao zerar a janela a cada perda.
- **Reno** ganhou o **Fast Recovery**, recuperando bem perdas isoladas — porém quebra diante de múltiplas perdas por janela.
- **NewReno** consertou o Fast Recovery com **ACKs parciais**, tratando múltiplas perdas sem suporte do receptor, ainda que ao ritmo de ~1 perda por RTT.
- **Vegas** trocou o paradigma: usa o **atraso** como sinal antecipado, reduzindo perdas e filas — ao custo de injustiça contra fluxos baseados em perda.
- **SACK** trocou a **informação**: reconhecimentos seletivos permitem recuperar vários buracos por RTT, tornando-se a base da recuperação eficiente moderna.

São, portanto, dois eixos de inovação: **melhorar a recuperação** (Tahoe → Reno → NewReno → SACK) e **melhorar o sinal de congestionamento** (perda → atraso, com o Vegas).

5.2 Qual é a mais robusta no cenário atual, e por quê?

Considerando as cinco variantes do escopo, a mais robusta no cenário atual é o **TCP SACK**. Três razões sustentam a escolha:

1. **Cobertura de cenários** — é a única que recupera **múltiplas perdas por janela em poucos RTTs**, o que a torna eficaz tanto em redes com perdas quanto em enlaces de alto produto banda-atraso (satélite, longa distância).
2. **Adoção real** — está **habilitada por padrão** em Linux, Windows e macOS; é o denominador comum das pilhas TCP modernas, o que a torna robusta na prática, e não só no papel.
3. **Composabilidade** — o SACK opera na camada de *recuperação de perdas* e se combina naturalmente com os algoritmos de *janela* do estado da arte (CUBIC, BBR), permanecendo relevante mesmo com a evolução do controle de congestionamento.

Em suma: das variantes clássicas, **NewReno** é a mais robusta que depende só do emissor, e **Vegas** aponta o futuro (controle por atraso, hoje materializado no BBR); mas o **SACK** é a resposta mais equilibrada e universal para as redes de hoje — inclusive 4G/5G e satélite.

6. Referências

As referências abaixo priorizam **fontes primárias** (RFCs e artigos originais) e duas obras de referência amplamente adotadas no ensino de redes.

RFCs (IETF)

1. RFC 793 — *Transmission Control Protocol*. IETF, 1981. (Atualizada pela RFC 9293, 2022.)
2. RFC 2001 — Stevens, W. *TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms*. IETF, 1997.
3. RFC 2581 — Allman, M.; Paxson, V.; Stevens, W. *TCP Congestion Control*. IETF, 1999.
4. RFC 5681 — Allman, M.; Paxson, V.; Blanton, E. *TCP Congestion Control*. IETF, 2009.
5. RFC 2582 / RFC 3782 / RFC 6582 — Floyd, S.; Henderson, T.; Gurtov, A. *The NewReno Modification to TCP's Fast Recovery Algorithm*. IETF, 1999 / 2004 / 2012.
6. RFC 2018 — Mathis, M.; Mahdavi, J.; Floyd, S.; Romanow, A. *TCP Selective Acknowledgment Options*. IETF, 1996.
7. RFC 2883 — Floyd, S.; Mahdavi, J.; Mathis, M.; Podolsky, M. *An Extension to the Selective Acknowledgement (SACK) Option for TCP*. IETF, 2000.
8. RFC 3517 / RFC 6675 — Blanton, E.; Allman, M.; et al. *A Conservative Loss Recovery Algorithm Based on Selective Acknowledgment (SACK) for TCP*. IETF, 2003 / 2012.
9. RFC 6298 — Paxson, V.; Allman, M.; Chu, J.; Sargent, M. *Computing TCP's Retransmission Timer*. IETF, 2011.
10. RFC 8312 — Rhee, I.; Xu, L.; Ha, S.; et al. *CUBIC for Fast Long-Distance Networks*. IETF, 2018.

Artigos

11. Jacobson, V. *Congestion Avoidance and Control*. Proc. ACM SIGCOMM, 1988.

12. Brakmo, L. S.; O'Malley, S. W.; Peterson, L. L. *TCP Vegas: New Techniques for Congestion Detection and Avoidance*. Proc. ACM SIGCOMM, 1994. (Versão estendida: *IEEE Journal on Selected Areas in Communications*, v. 13, n. 8, 1995.)
13. Fall, K.; Floyd, S. *Simulation-based Comparisons of Tahoe, Reno, and SACK TCP*. ACM SIGCOMM Computer Communication Review, 1996.

Livros

14. Kurose, J. F.; Ross, K. W. *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. Pearson (8ª ed.), cap. 3 (Camada de Transporte).
15. Stevens, W. R. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley.
16. Peterson, L. L.; Davie, B. S. *Computer Networks: A Systems Approach*. Morgan Kaufmann.

Documento-fonte em Markdown. Para gerar o PDF de entrega, veja a seção "Gerar o PDF do relatório" no `README.md` do projeto (Pandoc + XeLaTeX, ou "Imprimir → Salvar como PDF" a partir do Markdown renderizado).